

Merge-Insertion Sort. Ford Jonshon's algorithm

Input
57 13 41 34 76 22 48 65 8 34 84 14 72 5 61 97 28 53 11 88 44

1 Grouping

[57 13] [41 34] [76 22] [48 65] [8 34] [84 14] [72 5] [61 97] [28 53] [11 88] [44]

2 Sort the larger elements of each pair

[13 57] [34 41] [22 76] [48 65] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44] =>

=> [13 57] [34 41] [22 76] [48 65] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44] 10 comparisons

If the number in the pair is bigger than the other one, we swap them in place

[13 57] [34 41] [48 65] [22 76] [14 84] [19 84] [5 72] [61 97] [28 53] [11 88] [44] =>

=> [13 57] [34 41] [48 65] [22 76] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44] Level 1 recursion Merge Sort

The labels placed against the last number of each element

[13 57] [22 76] [13 57] [34 41] [48 65] [14 84] [5 72] [61 97] [28 53] [11 88] [44] =>

=> [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44] Level 2 recursion

[[48 65] [22 76]] [[13 57] [34 41]] [[8 34] [14 84]] [[5 72] [61 97]] Swap 4 elements

[[48 65] [22 76]] [[13 57] [34 41]] [[8 34] [14 84]] [[5 72] [61 97]] Swap 8 elements. But already sorted

[[48 65] [22 76]] [[13 57] [34 41]] [[8 34] [14 84]] [[5 72] [61 97]] Level 3 recursion

The recursion stops when we aren't able to form any more pairs of elements

4 Initialising and Insertion - Unwinding the Recursion

Two handy meanings to understand before:
The main chain - the smallest element of the smallest pair (b1) and the rest of a.
The pend chain - the rest of b starting from b2, and odd numbers if any from the (2) step.

Level 3 unwinding [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The main chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The pend chain [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

• Pend is empty, two elements go directly to main, because they are a part of it b1 and a1.

Level 2 unwinding [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The main chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The pend chain [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The manipulations above were initialising of the main and pend chain. As soon as we get the pend chain filled. Here comes the insertion of it to the main chain using binary search algorithm

The insertion order from the pend dictated by the Jacobsthal Sequence numbers

current = previous + 2*(previous of the previous) - formula for Jacobsthal sequence

0, 1, 1, 3, 5, 11, 21, 43, 85

The current Jacob number is 3, since it is the only one biggest b we have in pend.

The step is to insert b3 among {b1, a1, a2}, then b2 among the other elements less than a2.

Compare b3 to element n / 2 + 1, where n is the elements on the main chain. =>

[28 53] [11 88] < [13 57] [34 41] First comparison The main chain

[28 53] [11 88] < [13 57] [34 41] Second comparison [13 57] [34 41] [28 53] [11 88] [5 72] [61 97] [44]

The current Jacob number is still 3, the next element for insertion is b2. =>

Compare b2 to element n / 2 + 1, where n is the elements on the main chain = selects higher bound. Since we know that b2 less than a2, we don't take a2 into calculation. n = 3(not 4). =>

[8 34] [14 84] < [28 53] [11 88] First comparison The main chain

[8 34] [14 84] < [28 53] [11 88] Second comparison [13 57] [34 41] [28 53] [11 88] [5 72] [61 97] [44]

Level 1 unwinding [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The main chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

The pend chain [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

Jacobsthal number is 3, start with b3. Compare b3 to a1(n / 2) =>

[28 53] [11 88] < [22 76] First comparison [28 53] [11 88] < [48 65] Second comparison

Jacobsthal number is 3, continue with b2 => Max 2 comparisons in the worst case.

The MAIN chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

Jacobsthal number is 5, start with b5. Compare b5 to a1 (n / 2). => Max of 3 comparisons

[5 72] [22 76] First comparison [5 72] [22 76] Second comparison [5 72] [48 65] Third comparison

The MAIN chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

Jacobsthal number is 5, continue with b4. Compare b4 to b3. We need to compare b4 to the left part from the b5, since we know that b4 for sure smaller than b5.

[13 57] [22 76] First comparison [13 57] [22 76] Second comparison

The MAIN chain [13 57] [22 76] [13 57] [34 41] [8 34] [14 84] [5 72] [61 97] [28 53] [11 88] [44]

Finalise with insertion of b at pend chain into a at main chain

[8 34] [28 53] [13 57] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

The MAIN chain [8 34] [28 53] [13 57] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

The pend chain [28 53] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

First b3, b2, b5, b4, b1, b10, b9, b7, b6.

[8 34] [28 53] [13 57] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

The MAIN chain [8 34] [28 53] [13 57] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

The pend chain [28 53] [48 65] [5 72] [22 76] [14 84] [11 88] [34 41] [61 97] [44]

Way too lazy to explain it for you. The algo of insertion is completely same as in th recursion unwinding(4).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

[8 34] [28 53] [13 57] [48 65] [5 72] ... • You compare the jacobsthal number to the element n/2, depending on the result compare the number on the side to the n/2. n/2 in this case is the amount of element from the b1(include) to corresponding a(exclude).

Calculate Number of Comparisons

• (1) & (2) => takes at most n / 2 number of comparisons, not matter if even or odd.

• (3) & (4) => takes at most 22 number of comparisons for n/2 = 10

level 1 recursion = at most 5 comparisons, since 5 pairs of pairs

level 2 recursion = at most 2 comparisons, since 4 pairs of pairs of pairs

level 3 recursion = at most 1 comparison, since 2 pairs

level 3 unwinding = no comparisons

level 2 unwinding = insert b3 & b2, at most 2 + 2=4 comparisons

level 1 unwinding = insert b3 & b2, b5 & b4, = 2 + 2 + 3 + 3 = 10 comparis

total => S(n/2) = 5 + 2 + 1 + 4 + 10 = 22.

Pairing (10) + {Sort 10 winners} [F(10) = 22] + {Insert remaining} (34) = 66 comparisons.

n = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

[ln n!] = 0 1 3 5 7 10 13 16 19 22 26 29 33 37 41 45 49